

JavaScript Numbers

[Back to JS page](#)

Table of Contents

- [JavaScript Numbers](#)
 - [Number Types](#)
 - [Example 1](#)
 - [Integer Precision](#)
 - [Example 2](#)
 - [Adding Numbers and Strings](#)
 - [Example 3](#)
 - [Numeric Strings](#)
 - [Example 4](#)
 - [NaN - Not a Number](#)
 - [Example 5](#)
 - [Infinity](#)
 - [Example 6](#)
 - [Hexadecimal](#)
 - [Example 7](#)
 - [JavaScript Numbers as Objects](#)
 - [Example 8](#)
 - [Document](#)
 - [Reference](#)

Number Types

JavaScript has only one type of number. Numbers can be written with or without decimals:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Number Types</h4>
<p id="demo"></p>

<script>
let x = 3.14;    // A number with decimals
let y = 3;      // A number without decimals
let z = 123e5;   // Exponential notation
document.getElementById("demo").innerHTML =
  x + "<br>" + y + "<br>" + z;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Integer Precision

Integers (numbers without a period or exponent notation) are accurate up to 15 digits:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Integer Precision</h4>
<p id="demo"></p>

<script>
let x = 9999999999999999;    // 15 digits
let y = 99999999999999999;  // 16 digits
document.getElementById("demo").innerHTML =
    "15 digits: " + x + "<br>" +
    "16 digits: " + y;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Adding Numbers and Strings

When adding numbers and strings, JavaScript treats numbers as strings when a string is present:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Adding Numbers and Strings</h4>
<p id="demo"></p>

<script>
let x = 10;
let y = 20;
let z = "The result is: " + x + y;
document.getElementById("demo").innerHTML = z;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Numeric Strings

JavaScript strings can have numeric content. JavaScript will try to convert strings to numbers in some operations:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Numeric Strings</h4>
<p id="demo"></p>

<script>
let x = "100";
let y = "10";
let z = x / y;
document.getElementById("demo").innerHTML = "100 / 10 = " + z;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

NaN - Not a Number

NaN is a JavaScript reserved word indicating that a number is not a legal number:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>NaN - Not a Number</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let x = 100 / "Apple";
document.getElementById("demo1").innerHTML = "100 / 'Apple' = " + x;
document.getElementById("demo2").innerHTML = "isNaN(100 / 'Apple'): " + isNaN(x);
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Infinity

Infinity (or **-Infinity**) is the value JavaScript will return if you calculate a number outside the largest possible number:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Infinity</h4>
<p id="demo"></p>

<script>
let x = 2 / 0;
let y = -2 / 0;
document.getElementById("demo").innerHTML =
  "2 / 0 = " + x + "<br>" +
  "-2 / 0 = " + y + "<br>" +
  "typeof Infinity: " + typeof x;
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Hexadecimal

JavaScript interprets numeric constants as hexadecimal if they are preceded by `0x` :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Hexadecimal</h4>
<p id="demo"></p>

<script>
let x = 0xFF;
document.getElementById("demo").innerHTML = "0xFF = " + x;
</script>

</body>
</html>
```

Example 7

Result [View Example](#)

JavaScript Numbers as Objects

Normally, JavaScript numbers are primitive values created from literals.

But numbers can also be defined as objects with the keyword `new` :


```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Numbers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Numbers as Objects</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
let x = 123;
let y = new Number(123);
document.getElementById("demo1").innerHTML = typeof x + "<br>" + typeof y;
document.getElementById("demo2").innerHTML = "x === y is " + (x === y);
</script>

</body>
</html>
```

Example 8

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Numbers](#)